Disjoint Set Union (DSU) Reference

Contents

1 Basic DSU (Union by Size + Path Compression)

Standard Disjoint Set Union with union by size and path compression. `par[u] < 0` means `u` is a root and the component size is `-par[u]`.

```cpp
#include <bits/stdc++.h>
using namespace std;

#define nl "\n"
#define sp " "

const int N = 100005;
struct DSU {
    // par[u] < 0  ->  root, component size = -par[u]
    // par[u] >= 0 ->  non-root, parent = par[u]
    int par[N], ncmp;
    void init(int n) {
        memset(par, -1, n * sizeof par[0]); // -1 = root of size 1
        ncmp = n;
    }

    int find(int u) {
        return par[u] < 0 ? u : par[u] = find(par[u]); // path compression
    }

    // s = small, b = big (by convention - swap if inconsistent).
    // Returns false if s and b are already in the same component.
    bool join(int s, int b) {
        s = find(s); b = find(b);
        if (s == b) return false;
        if (par[b] > par[s]) swap(s, b); // par more negative = bigger; ensure b
             is bigger
        par[b] += par[s];                // merge sizes (both negative -> sum
            more negative)
        par[s] = b;                      // s becomes child of b
        ncmp--;
        return true;
    }
};

int main() {
    int n, m;
    cin >> n >> m;

    DSU dsu;
    dsu.init(n); // 0-based

    int u, v;
    for(int i = 0; i < m; i++) {
        cin >> u >> v;
        dsu.join(u, v);
    }

    // How many connected components remain?
    cout << "Components: " << dsu.ncmp << nl;

    // Check if two specific nodes are connected
    int a = 0, b = 1;
    if (dsu.find(a) == dsu.find(b))
        cout << a << " and " << b << " are in the same component" << nl;
    else
        cout << a << " and " << b << " are in different components" << nl;

    // Get the size of the component containing a given node q
    int q = 0;
    int root = dsu.find(q);
```

```
    int size = -dsu.par[root];
    cout << "Component of " << q << " has size " << size << nl;
}
```

2 Parity DSU (Bipartite / 2-Coloring)

Maintains parity (XOR distance) to the root. Useful for checking bipartiteness or enforcing "same group / different group" constraints.

```
#include <bits/stdc++.h>
using namespace std;

#define nl "\n"
#define sp " "

const int N = 100005;

struct ParityDSU {
    int par[N], dist[N]; // dist[u] = XOR parity to root
    void init(int n){
        fill(par, par+n, -1);
        fill(dist, dist+n, 0);
    }

    // Returns {root, distance_to_root}
    pair<int,int> find(int u){
        if(par[u] < 0) return {u, 0}; // par[u]<0 means u is root
        auto [root, d] = find(par[u]);
        dist[u] ^= dist[par[u]]; // path compression + parity
        par[u] = root;
        return {root, dist[u]};
    }

    // p=0: u and v same group, p=1: different group
    // Returns: -1=conflict, 0=new edge ok, 1=redundant ok
    int addEdge(int u, int v, int p){
        auto [ru, du] = find(u);
        auto [rv, dv] = find(v);
        if(ru == rv){
            return (du ^ dv) == p ? 1 : -1; // check consistency
        }

        // Union by size
        if (par[ru] > par[rv]) {
            swap(ru, rv);
            swap(du, dv);
        }
        par[ru] += par[rv]; // Merge the size of rv into ru

        par[rv] = ru; // rv (smaller) becomes child of ru (bigger)
        dist[rv] = du ^ dv ^ p; // set parity so dist[v] remains correct
        return 0;
    }
};

int main() {
    ParityDSU pdsu;
    pdsu.init(5);

    int r1 = pdsu.addEdge(0, 1, 1); // 0 and 1 differ
    cout << "addEdge(0,1,differ): " << r1 << " (0=new edge)" nl;

    int r2 = pdsu.addEdge(1, 2, 1); // 1 and 2 differ -> implies 0,2 same
    cout << "addEdge(1,2,differ): " << r2 << " (0=new edge)" nl;

```

```
    int r3 = pdsu.addEdge(0, 2, 0); // check: 0 and 2 same group -> should be
        consistent
    cout << "addEdge(0,2,same): " << r3 << " (1=redundant, consistent)" nl;

    int r4 = pdsu.addEdge(0, 2, 1); // conflicting claim: 0 and 2 differ
    cout << "addEdge(0,2,differ): " << r4 << " (-1=conflict)" nl;
}
```

3 Weighted DSU (Difference Constraints)

Maintains relative values between nodes. Constraint form: `value[v] - value[u] = d`.

```
#include <bits/stdc++.h>
using namespace std;

#define nl "\n"
#define ll long long
#define sp " "

const int N = 100005;

struct WeightedDSU {
    int par[N];
    long long w[N]; // w[u] = value[u] - value[parent_of_u]

    void init(int n) {
        fill(par, par + n, -1);
        fill(w, w + n, 0);
    }

    // Returns {root, value[u] - value[root]}
    pair<int, long long> find(int u) {
        if (par[u] < 0) return {u, 0};
        auto [root, wr] = find(par[u]);
        w[u] += wr; // path compression: accumulate weight
        par[u] = root;
        return {root, w[u]};
    }

    // Constraint: value[v] - value[u] = d
    // Returns false if it contradicts existing information
    bool join(int u, int v, long long d) {
        auto [ru, wu] = find(u); // wu = value[u] - value[ru]
        auto [rv, wv] = find(v); // wv = value[v] - value[rv]

        if (ru == rv) {
            return (wv - wu) == d; // check consistency
        }

        // Union by size     ensure ru is the larger component
        if (par[ru] > par[rv]) {
            swap(ru, rv);
            swap(wu, wv);
            d = -d; // Because constraint was v-u=d, swapping makes it u-v=-d
        }
        par[ru] += par[rv];

        par[rv] = ru;
        w[rv] = wu - wv + d; // so that value[v]-value[u] = d holds
        return true;
    }

    // Query: value[v] - value[u] (must be in same component)
    long long query(int u, int v) {
        auto [ru, wu] = find(u);
```

```
        auto [rv, wv] = find(v);
        if (ru != rv) return LLONG_MIN; // flag for "not connected"
        return wv - wu;
    }
};

int main() {
    WeightedDSU wdsu;
    wdsu.init(5);

    bool ok1 = wdsu.join(0, 1, 5); // value[1]-value[0] = 5
    cout << "join(0,1,5): " << ok1 << " (1=ok)" nl;

    bool ok2 = wdsu.join(1, 2, 3); // value[2]-value[1] = 3
    cout << "join(1,2,3): " << ok2 << " (1=ok)" nl;

    ll q1 = wdsu.query(0, 2); // expect 8
    cout << "query(0,2) = " << q1 << " (expected 8)" nl;

    bool ok3 = wdsu.join(0, 2, 8); // consistent with derived value
    cout << "join(0,2,8): " << ok3 << " (1=ok, consistent)" nl;

    bool ok4 = wdsu.join(0, 2, 100); // contradicts derived value
    cout << "join(0,2,100): " << ok4 << " (0=conflict)" nl;
}
```

4 DSU with Forward-Star Component Listing

Keeps an explicit linked list of every member of each component so you can retrieve the full list of nodes in $O(size)$ time.

```
#include <bits/stdc++.h>
using namespace std;

#define nl "\n"
#define sp " "

const int N = 100005;
struct DSU {
    // par[u] < 0  -> root, component size = -par[u]
    // par[u] >= 0 -> non-root, parent = par[u]
    int par[N], ncmp, sets[N], pos[N];
    int tail[N], nxt[N];
    void init(int n) {
        memset(par, -1, n * sizeof par[0]); // -1 = root of size 1
        iota(sets, sets + n, 0);            // active root list
        iota(pos, pos + n, 0);              // pos[u] = index of u in sets[]
        iota(tail, tail + n, 0);            // tail[u] = last node in u's list
        ncmp = n;
        memset(nxt, -1, n * sizeof nxt[0]);
    }

    int find(int u) {
        return par[u] < 0 ? u : par[u] = find(par[u]); // path compression
    }

    // Returns false if s and b are already in the same component.
    bool join(int s, int b) {
        s = find(s); b = find(b);
        if (s == b) return false;
        if (par[b] > par[s]) swap(s, b); // ensure b is bigger
        par[b] += par[s];
        par[s] = b;

        // link the two component lists
```

```
        int &t = tail[b];
        nxt[t] = s; t = tail[s];

        // remove the smaller root from the active-root list
        int p = pos[s];
        sets[p] = sets[--ncmp];
        pos[sets[p]] = p;

        return true;
    }

    // Returns all nodes in the component containing u.
    vector<int> getCmp(int u) {
        u = find(u);
        vector<int> ret;
        ret.reserve(-par[u]);
        for (int v = u; v != -1; v = nxt[v])
            ret.push_back(v);
        return ret;
    }

    // Returns all components as a list of node lists.
    vector<vector<int>> getCmps() {
        vector<vector<int>> ret;
        ret.reserve(ncmp);
        for (int i = 0; i < ncmp; i++)
            ret.push_back(getCmp(sets[i]));
        return ret;
    }
};

int main() {
    int n, m;
    cin >> n >> m;

    DSU dsu;
    dsu.init(n); // 0-based

    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        dsu.join(u, v);
    }

    cout << "Components: " << dsu.ncmp << nl;

    int a = 0, b = 1;
    if (dsu.find(a) == dsu.find(b))
        cout << a << " and " << b << " are in the same component" << nl;
    else
        cout << a << " and " << b << " are NOT in the same component" << nl;

    vector<int> comp0 = dsu.getCmp(0);
    cout << "Component of 0 has " << comp0.size() << " nodes: ";
    for (int x : comp0) cout << x << sp;
    cout << nl;

    vector<vector<int>> all = dsu.getCmps();
    for (auto &c : all) {
        cout << "Component: ";
        for (int x : c) cout << x << sp;
        cout << nl;
    }
}
```

5 DSU with Rollback

Supports undoing unions (useful for offline queries, segment-tree + DSU, etc.). Path compression is also logged so it can be reversed.

```cpp
#include <bits/stdc++.h>
using namespace std;

#define nl "\n"
#define sp " "

const int N = 100005;
const int M = 100005;

struct DSU
{
    // par[u] < 0  -> root, size = -par[u]
    // par[u] >= 0 -> non-root, parent = par[u]
    int par[N], curT, curTtoSs[M];
    void init(int n)
    {
        curT = 0;
        memset(par, -1, n * sizeof par[0]); // -1 = root of size 1
    }

    // Each op saves {node, its old par, other node or -1, its old par}
    struct operation { int a, parA, b, parB; };
    stack<operation> ops;

    // Find with path compression (logged for rollback)
    int operator[](int u)
    {
        if(par[u] < 0) return u;
        ops.push({u, par[u], -1, 0});      // compression: only par[u] changes
        return par[u] = (*this)[par[u]];
    }

    // Join (union by size)
    bool operator()(int a, int b)
    {
        curTtoSs[curT++] = ops.size();
        a = (*this)[a];
        b = (*this)[b];
        if(a == b) return 0;
        if(par[a] > par[b]) swap(a, b);    // a has bigger size
        ops.push({b, par[b], a, par[a]}); // save BOTH roots
        par[a] += par[b];
        par[b] = a;
        return 1;
    }

    void rollBack()
    {
        auto &op = ops.top();
        par[op.a] = op.parA;
        if(~op.b) par[op.b] = op.parB;     // restore other root if union
        ops.pop();
    }

    void rollUntil(int t)
    {
        while(ops.size() > curTtoSs[t + 1])
            rollBack();
        curT = t + 1;
    }
} dsu;
```

```
62
63 int main() {
64     int n = 5;
65     dsu.init(n); // 0-based
66
67     dsu(0, 1);
68     dsu(1, 2);
69     dsu(3, 4);
70
71     // Roll back to the state right after call 0 (undoes calls 1 and 2)
72     dsu.rollUntil(0);
73     cout << "After rollUntil(0):" nl;
74     cout << "  0 and 1 connected? " << (dsu[0] == dsu[1] ? "yes" : "no") << nl;
75     cout << "  0 and 2 connected? " << (dsu[0] == dsu[2] ? "yes" : "no") << nl;
76     cout << "  3 and 4 connected? " << (dsu[3] == dsu[4] ? "yes" : "no") << nl;
77 }
```